

Université de Lorraine - UFR Sciences Fondamentales et Appliquées

Compte rendu du TP architectures embarquées :

Home Assistant - Station météo connectée autonome

Groupe HTML / API JavaScript

Par :

Nom : BELKACEM

Prénom : Mohamed Amine

Université de Lorraine – Année 2025-2026

Table des matières

1. Introduction 3
2. Organisation du projet et planification (Gantt) 3
3. Architecture globale du système 4
   3.1 Schéma d'architecture 4
   3.2 Diagramme de séquence 4
4. Comment la page communique avec l'API 5
   4.1 Les trois étapes de la communication API ↔ HTML 6
   4.2 L'API externe Open-Meteo (météo officielle) 6
5. La partie REST 6
   5.1 Schéma de l'architecture REST 7
6. Pourquoi JavaScript et pas PHP ? 7
7. Optimisation du code (HTML / CSS / JavaScript) 8
   7.1 Optimisation HTML 8
   7.2 Optimisation CSS 8
   7.3 Optimisation JavaScript 8
8. Gestion de versions avec GitHub 9
9. Résultats et tests 10
10. Conclusion 10
11. Annexe - Codes complets 12
   11.1 Code HTML optimisé (index.html) 12
   11.2 Code CSS optimisé (style.css) 14
   11.3 Code JavaScript optimisé (app.js) 22

Liste des figures

Figure 1 : Schéma d'architecture 4
Figure 2 : Diagramme de séquence 5
Figure 3 : Architecture REST 7
Figure 4 : Dashboard station météo finale 10

1. Introduction

Dans ce projet, l'objectif était de concevoir un système complet permettant de collecter, traiter et afficher des données issues de différents capteurs connectés. Le travail était réparti en plusieurs groupes : un groupe chargé de la programmation de l'ESP32 et des capteurs, un autre responsable de la configuration du Raspberry Pi et de l'intégration avec Home Assistant, et le groupe HTML, dont je faisais partie, dédié au développement de l'interface web.

Mon rôle dans ce projet consistait principalement à assurer la liaison entre les données générées par les capteurs et leur affichage sur une page web fluide et réactive. Pour cela, la page s'appuie sur l'API REST (Representational State Transfer) fournie par Home Assistant. Mais surtout, mon rôle principal a été d'optimiser tout le code HTML, CSS et surtout JavaScript pour assurer un fonctionnement fluide du système.

Les capteurs envoient leurs données à un microcontrôleur ESP32 (bus I2C pour les capteurs BME280, BMP180 et BH1750, broches GPIO pour le vent et la pluie), qui les transmet ensuite par Wi-Fi au Raspberry Pi. Home Assistant centralise ces données et les expose à travers son API REST. Côté page web, j'utilise la fonction `fetch()` pour envoyer des requêtes HTTP à cette API, recevoir les données au format JSON, puis les afficher dynamiquement à l'utilisateur. La page est donc un client de l'API de Home Assistant : je n'ai pas écrit d'API serveur moi-même.

2. Organisation du projet et planification (Gantt)

Le projet a été mené par 10 étudiants. La planification a été formalisée dans un diagramme de Gantt commun.

Le travail était réparti en quatre groupes :

- **Groupe HTML** (dont je faisais partie) : développement de la page web (HTML/CSS/JS), intégration de l'API REST de Home Assistant, optimisation du front-end.
- **Groupe ESP32** : programmation du microcontrôleur ESP32, lecture des capteurs (BME280, BMP180, BH1750, vent, pluie), montage de la station.
- **Groupe Home Assistant** : installation et configuration de Home Assistant sur Raspberry Pi 5, intégration ESPHome, configuration YAML.
- **Maquette du site** : croquis du site, définition des objectifs visuels, organisation du Gantt global.

De mon côté, mes tâches principales ont été :

1. L'optimisation du code HTML, CSS et surtout JavaScript.
2. Le développement du code HTML/CSS et du code JavaScript qui interroge l'API, en me concentrant sur la communication avec Home Assistant.
3. La compréhension de l'architecture REST (GET, POST, PUT, DELETE).
4. L'aide aux tests et à l'intégration des capteurs BME280 et BH1750 et de la station météo Bresser.
5. La création et l'organisation du dépôt GitHub du projet : mise en place du dépôt `interface_station_meteo`, structuré en trois versions (v1.0, v2.0, v3.0) dans des dossiers séparés pour bien suivre l'évolution du projet.

3. Architecture globale du système

Le système suit une chaîne IoT classique en 4 niveaux. Au plus bas niveau, on trouve les capteurs physiques : BME280 pour la température, l'humidité et la pression, BMP180 pour la pression barométrique, BH1750 pour la luminosité, et la station Bresser pour le vent et la pluie. Les capteurs BME280, BMP180 et BH1750 sont reliés à l'ESP32 par le bus I2C, la station vent/pluie par des broches GPIO. L'ESP32, programmé via ESPHome, transmet ensuite les valeurs lues par Wi-Fi vers le Raspberry Pi 5.

Le Raspberry Pi héberge Home Assistant (HA), qui centralise tous les états des entités. HA expose une API REST pour lire ces états (et aussi une API WebSocket, que je n'ai pas utilisée dans cette version). Ma page web (HTML/CSS/JS) fait un `GET /api/states` au chargement pour afficher les valeurs, puis répète cette requête toutes les 5 secondes (polling) pour rester à jour. En parallèle, elle interroge l'API Open-Meteo pour afficher la météo officielle au même endroit.

3.1 Schéma d'architecture

Le schéma ci-dessous représente la circulation des données depuis les capteurs jusqu'à l'utilisateur final.

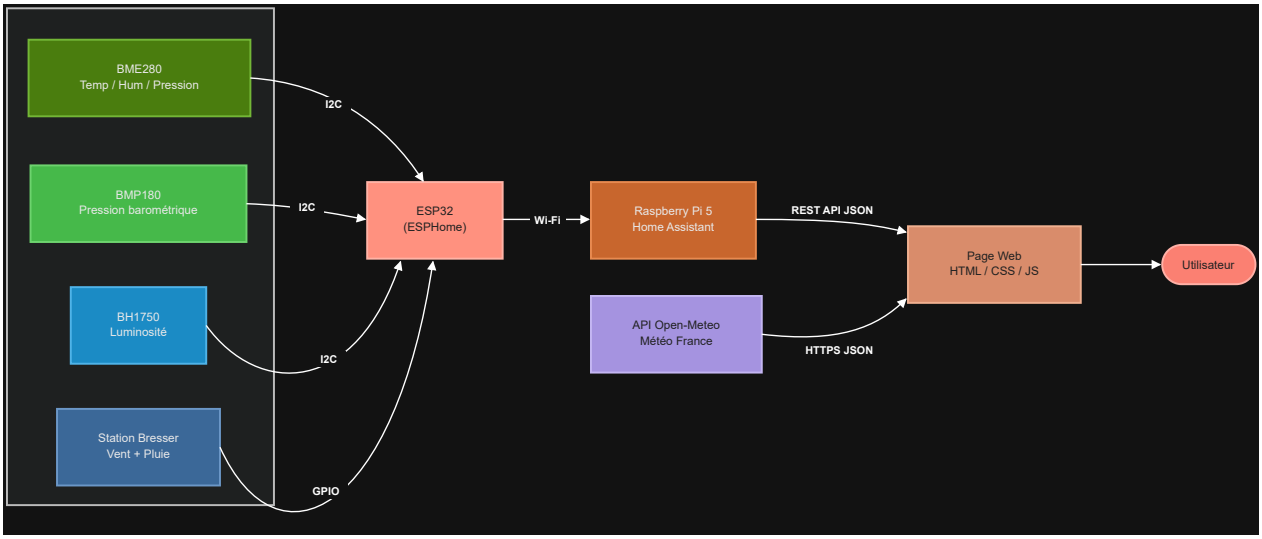


Figure 1 : Schéma d'architecture

3.2 Diagramme de séquence

Ce diagramme montre l'ordre des échanges entre la page web, l'API Home Assistant, les capteurs et l'API Open-Meteo, depuis l'ouverture de la page jusqu'aux mises à jour périodiques.

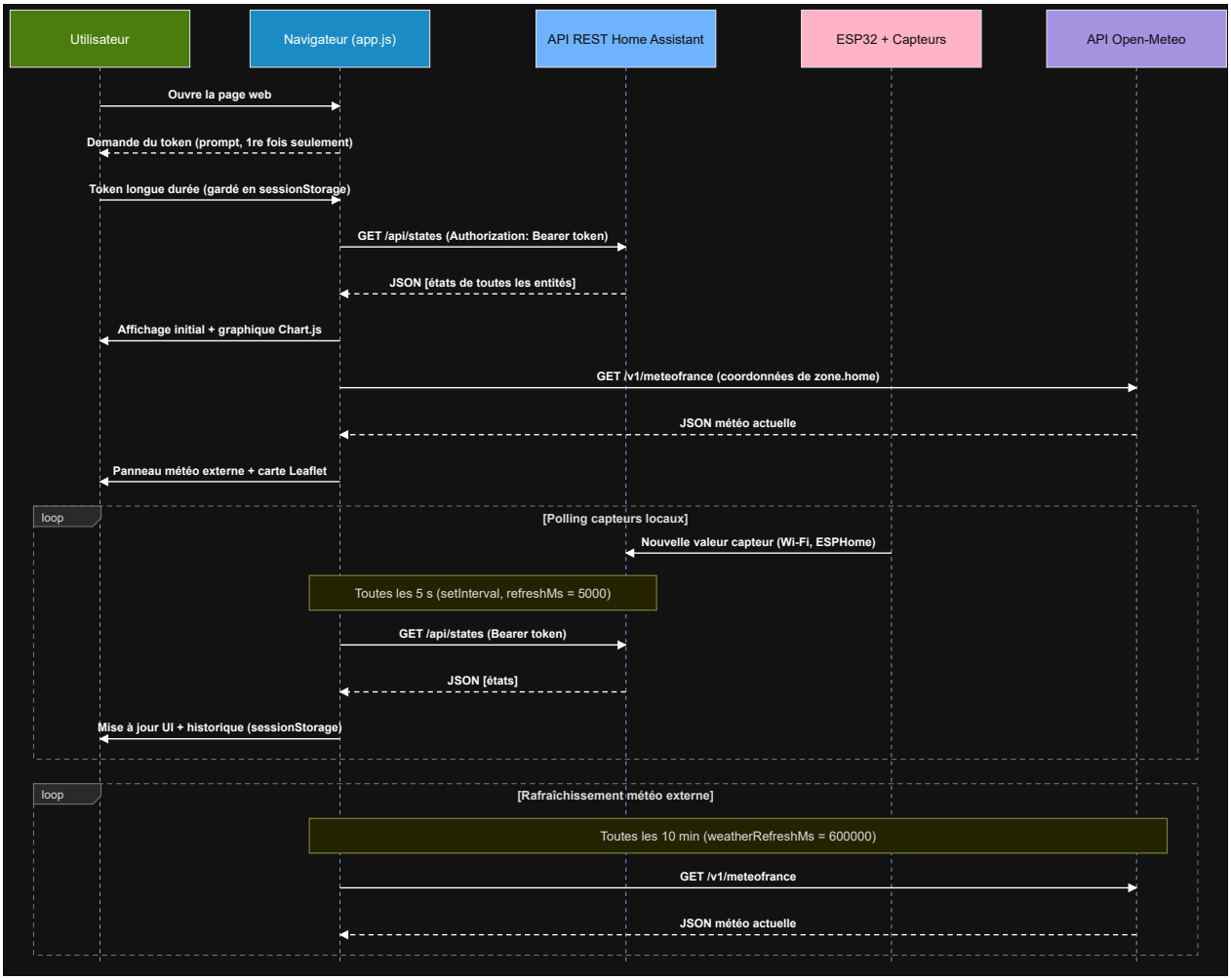


Figure 2 : Diagramme de séquence

4. Comment la page communique avec l'API

La page HTML (côté client) envoie des requêtes à l'API (côté serveur), puis récupère les données pour les afficher. Une page HTML seule ne peut pas lire directement l'état des capteurs dans Home Assistant : elle a besoin d'un intermédiaire, et cet intermédiaire est l'API REST de Home Assistant. Elle fait le pont entre le serveur (où se trouvent les données des capteurs) et la page web. J'utilise JavaScript dans la page HTML pour envoyer des requêtes HTTP vers cette API, avec la méthode GET pour récupérer les données.

L'API reçoit la requête, la traite, puis renvoie une réponse au format JSON. Ensuite, côté page, JavaScript récupère ces données et les injecte directement dans le HTML. Cela permet d'afficher les valeurs des capteurs (température, humidité, etc.) sans recharger la page. Dans ma version, la page relance cette requête toutes les 5 secondes (polling). Un WebSocket serait plus efficace, puisque le serveur notifierait lui-même chaque changement au lieu d'être interrogé en boucle ; je le garde comme amélioration pour une prochaine version.

La fonction utilisée pour cette communication est `fetch()`. Elle envoie une requête à l'API et récupère la réponse. Quand les données sont reçues, je les transforme en objet JavaScript avec `res.json()`, puis je les affiche dans des éléments HTML comme des `<div>`.

4.1 Les trois étapes de la communication API ↔ HTML

- **Étape 1** : la page envoie une requête HTTP à l'API (avec un token d'authentification dans le header `Authorization`).
- **Étape 2** : l'API REST de Home Assistant traite la demande, lit l'état des entités et renvoie une réponse JSON.
- **Étape 3** : la page parse le JSON et l'injecte dans les éléments HTML, ce qui met à jour l'affichage sans recharger la page.

4.2 L'API externe Open-Meteo (météo officielle)

C'est une API gratuite et open source qui propose plusieurs sources de données météo, dont Météo-France pour l'Europe. Le gros avantage, c'est qu'elle ne demande ni clé API ni inscription, contrairement à OpenWeatherMap ou AccuWeather. C'est très pratique pour ce projet.

Je l'ai ajoutée pour pouvoir comparer en direct les valeurs mesurées par nos capteurs avec celles de Météo-France pour la même localisation. Par exemple, si le capteur BME280 indique 22 °C et que Météo-France annonce 24 °C, on peut détecter un éventuel décalage (capteur mal positionné, soleil direct, calibration à revoir, etc.).

5. La partie REST

La communication entre la page web et Home Assistant repose sur le principe REST (Representational State Transfer). C'est une architecture très utilisée pour permettre à différents systèmes de communiquer entre eux de manière simple et standardisée.

Une API REST fonctionne à l'aide de requêtes HTTP, chaque type ayant un rôle bien précis :

- **GET** : récupérer des données (par ex. les valeurs des capteurs). C'est la seule méthode utilisée par mon dashboard.
- **POST** : envoyer de nouvelles données (par ex. déclencher un service).
- **PUT** : modifier des données existantes.
- **DELETE** : supprimer des données.

Le principe REST impose aussi une organisation claire des routes (URL). Par exemple, dans Home Assistant :

- `/api/states` : récupérer l'état actuel de toutes les entités (route utilisée par mon dashboard).
- `/api/states/sensor.temperature_salon` : récupérer un capteur spécifique.
- `/api/history/period/{date}` : récupérer l'historique sur une période.
- `/api/services/light/turn_on` : déclencher un service (allumer une lumière, etc.).

5.1 Schéma de l'architecture REST

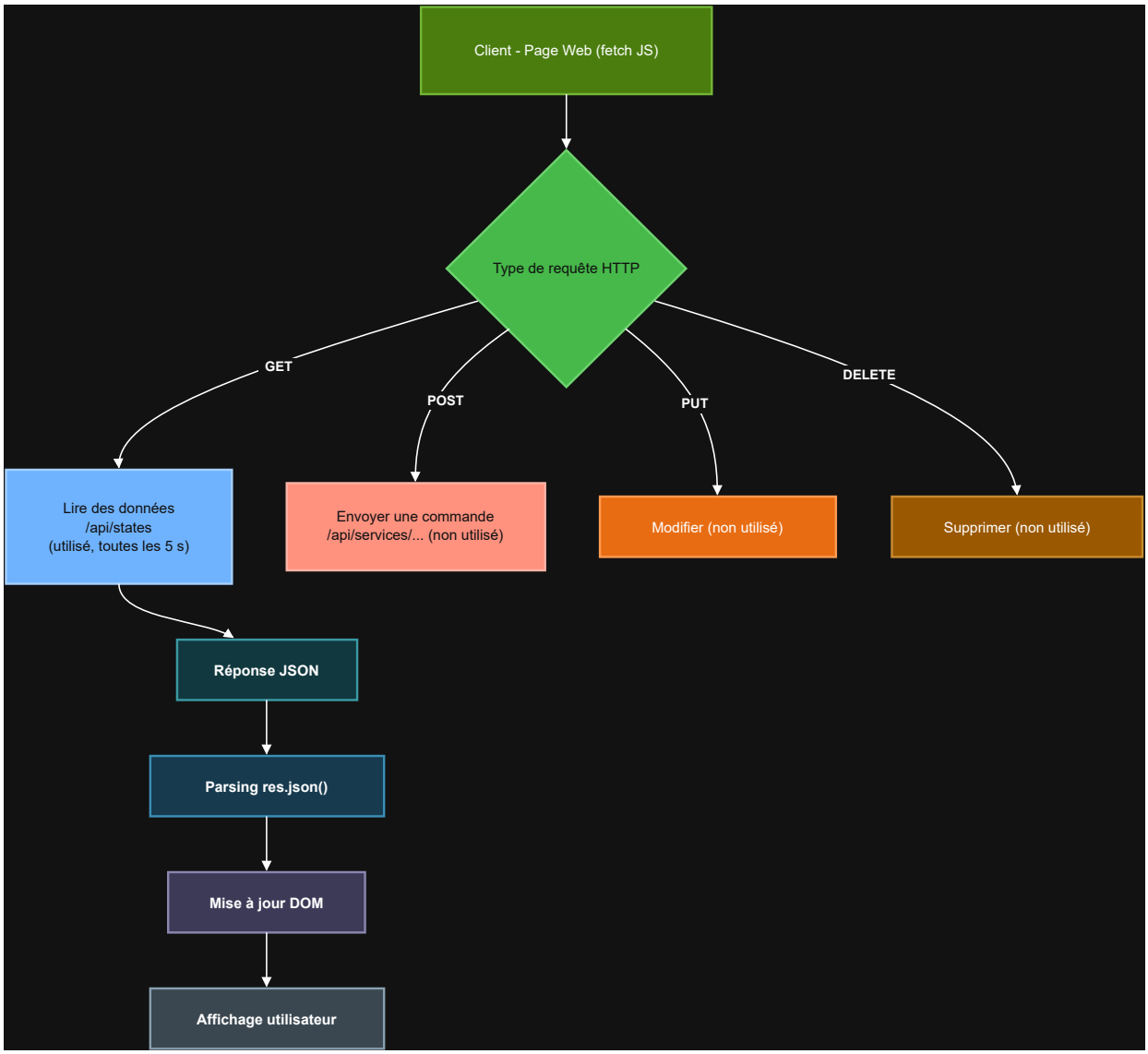


Figure 3 : Architecture REST

6. Pourquoi JavaScript et pas PHP ?

Je me suis posé la question au début du projet : pourquoi utiliser JavaScript pour interroger l'API, et pas PHP, qui est lui aussi très utilisé pour les sites web dynamiques ? Plusieurs raisons m'ont poussé à choisir JavaScript côté navigateur.

D'abord, PHP s'exécute sur un serveur. Pour l'utiliser ici, il aurait fallu installer et faire tourner un serveur PHP intermédiaire entre la page et Home Assistant. Or Home Assistant expose déjà une API REST : la page peut l'appeler directement depuis le navigateur avec `fetch()`, sans couche supplémentaire à maintenir. (PHP n'oblige pas pour autant à recharger la page : un script PHP peut lui aussi renvoyer du JSON à un `fetch()`.)

Ensuite, utiliser JavaScript partout rend le code plus cohérent et plus simple à maintenir : la logique d'appel à l'API, la mise à jour du DOM, le graphique Chart.js et la carte Leaflet sont dans le même langage. Enfin, le JSON renvoyé par l'API se manipule nativement en JavaScript, sans conversion.

7. Optimisation du code (HTML / CSS / JavaScript)

Une fois que la première version fonctionnait à peu près, j'ai consacré pas mal de temps à optimiser le code HTML, le CSS et surtout la partie JavaScript qui interroge l'API. Voici les principales optimisations que j'ai mises en place et leur justification.

7.1 Optimisation HTML

Côté HTML, j'ai apporté plusieurs améliorations qui paraissent petites mais qui ont un vrai intérêt en pratique :

- **Préconnexion aux 5 CDN/API externes** : j'ai ajouté des balises `<link rel="preconnect">` pour unpkg.com, jsdelivr.net, OpenStreetMap, api.open-meteo.com et geocoding-api.open-meteo.com. Le navigateur peut ainsi ouvrir les connexions TCP/TLS en avance, en parallèle du parsing HTML, au lieu d'attendre la première requête. Je n'ai pas mesuré le gain précis.
- **Scripts en `defer`** : tous les `<script>` sont passés en `defer`. Ils sont téléchargés en parallèle mais exécutés seulement après le parsing complet du DOM. Résultat : le contenu de la page s'affiche sans être bloqué par les scripts.
- **Accessibilité** : j'ai remplacé les `<div>` génériques par des balises sémantiques (`<main>`, `<section>`, `<header>`) et ajouté des attributs ARIA (`aria-live="polite"` sur les valeurs qui changent, `role="tablist"` sur les boutons, `aria-selected` sur le bouton actif). Un lecteur d'écran peut donc annoncer les changements de température sans interrompre l'utilisateur.

7.2 Optimisation CSS

Côté CSS, je voulais un design qui s'adapte à toutes les tailles d'écran :

- **Variables CSS** : les couleurs, espacements et tailles sont définis dans `:root` via des custom properties (`--bg1`, `--bg2`, `--bg3`, `--panel`, `--card`, `--text-soft`, etc.). Cela centralise le thème : on change une couleur à un seul endroit du fichier.
- **Tailles fluides avec `clamp()`** : les tailles de police utilisent `clamp(min, idéal, max)` pour s'adapter progressivement à la largeur de l'écran.
- **Responsive** : le design part de la mise en page bureau (grille 2 colonnes × 3 lignes) et l'adapte aux petits écrans (approche desktop-first). Une media query `@media (max-width: 1100px)` empile toutes les sections sur une seule colonne pour les tablettes, et `@media (max-width: 650px)` réorganise la barre du haut, les boutons et les cartes pour les smartphones. Une troisième, `@media (max-height: 930px)`, réduit les hauteurs sur les écrans d'ordinateur portable.
- **`prefers-reduced-motion`** : j'ai désactivé les transitions et animations pour les utilisateurs qui ont activé cette option (sensibilité au mouvement).
- **Focus visible** : j'ai ajouté `:focus-visible` sur les boutons et le champ de saisie pour que la navigation au clavier soit visible (contour bleu autour de l'élément sélectionné avec Tab).

7.3 Optimisation JavaScript

C'est sur cette partie que j'ai passé le plus de temps. La première version fonctionnait mais avait plusieurs problèmes : pas de gestion d'erreur sérieuse, token exposé en clair, requêtes qui pouvaient bloquer indéfiniment, et historique du graphique perdu à chaque rafraîchissement.

a) Sécurisation du token JWT

Dans la première version, le token longue durée de Home Assistant était écrit directement dans le JS. C'est un gros problème : le dépôt GitHub étant public, n'importe qui pouvait récupérer le token et accéder au Home Assistant. Dans la version optimisée, le token n'est plus jamais écrit dans le code : la page le demande une seule fois au chargement (`prompt()`) et le garde dans `sessionStorage`, qui est vidé à la fermeture de l'onglet. Les anciens tokens publiés doivent quand même être révoqués dans Home Assistant.

b) AbortController + timeout sur les requêtes fetch

Avant, si une API ne répondait pas, le `fetch` restait en attente sans limite. J'ai ajouté un `AbortController` avec un timeout de 8 secondes qui annule automatiquement la requête. Cela évite que la page reste bloquée sur une requête qui ne reviendra jamais. C'est appliqué à tous les `fetch` : Home Assistant, Open-Meteo et le géocodage.

c) Retry avec backoff exponentiel

Si une requête échoue, au lieu d'abandonner directement, le code la retente jusqu'à 2 fois, avec un délai qui double à chaque essai : 500 ms avant la 2e tentative, puis 1 s avant la 3e. Après la 3e tentative, l'erreur est affichée. C'est utile car les erreurs réseau sont souvent transitoires : un nouvel essai suffit souvent à passer outre.

d) Historique du graphique persisté dans sessionStorage

Avant, l'historique des capteurs (les points du graphique) était stocké uniquement en mémoire JavaScript. Si l'utilisateur rafraîchissait la page (F5), il perdait tout l'historique accumulé et le graphique repartait de zéro. Maintenant, l'historique est sauvegardé dans `sessionStorage` à chaque mise à jour et rechargé au démarrage : il survit à un F5, mais pas à la fermeture de l'onglet. Le graphique garde les 40 derniers points de chaque grandeur.

e) Map au lieu d'Object pour les états

Pour stocker les états des entités (avant : `let states = {}`), je suis passé à un `new Map()`. Les deux sont des tables de hachage avec un accès en O(1) en moyenne, donc le gain n'est pas une question de complexité. L'intérêt de Map est ailleurs : aucune clé héritée du prototype (un identifiant d'entité ne peut pas entrer en conflit avec une propriété comme `toString`), une méthode `get()` explicite et un ordre d'insertion garanti.

f) Animations Chart.js désactivées

Par défaut, Chart.js anime chaque mise à jour (les lignes bougent). C'est joli, mais cela consomme du CPU à chaque rafraîchissement. Avec `animation: false`, le graphique est redessiné directement, sans animation.

8. Gestion de versions avec GitHub

Pour la partie versionnage et collaboration, j'ai créé avec mon binôme du groupe HTML un dépôt GitHub nommé `interface_station_meteo`. C'était important pour plusieurs raisons : permettre à plusieurs personnes de travailler sur le code sans écraser le travail des autres, garder un historique propre des modifications, et pouvoir revenir à une version antérieure si on cassait quelque chose en cours de route.

9. Résultats et tests

Après plusieurs séances de tests et de corrections, le dashboard fonctionne correctement. Les valeurs des capteurs (température, humidité, pression, vent, luminosité, pluie) s'affichent à l'ouverture de la page, puis se mettent à jour toutes les 5 secondes sans rechargement grâce au polling de l'API REST.

La figure 4 montre le dashboard final pendant le TP. Elle permet de comparer, au même instant, les mesures de la station avec les données Open-Meteo pour la localisation de Home Assistant :

Grandeur	Capteur (Home Assistant)	Open-Meteo (Météo-France)
Température	24,67 °C	21,5 °C
Humidité	21,78 %	22 %
Pression	995,80 hPa	996,7 hPa
Vent	0,96 km/h	23,0 km/h

La pression et l'humidité sont proches des valeurs officielles. L'écart sur la température et surtout sur le vent vient probablement de l'emplacement de la station pendant le test (Open-Meteo donne des valeurs en extérieur dégagé) ; je n'ai pas eu le temps de le vérifier. C'est exactement le type de décalage que cette comparaison devait permettre de repérer.

Quelques difficultés rencontrées en cours de projet : la première a été la configuration du token. Au début, j'avais utilisé un token d'accès classique au lieu d'un token longue durée, donc il expirait toutes les 30 minutes. La deuxième a été la gestion des entités quand HA les renomme automatiquement (par exemple, `sensor.temperature_salon` devient parfois `sensor.temperature_salon_2` quand il y a plusieurs capteurs).

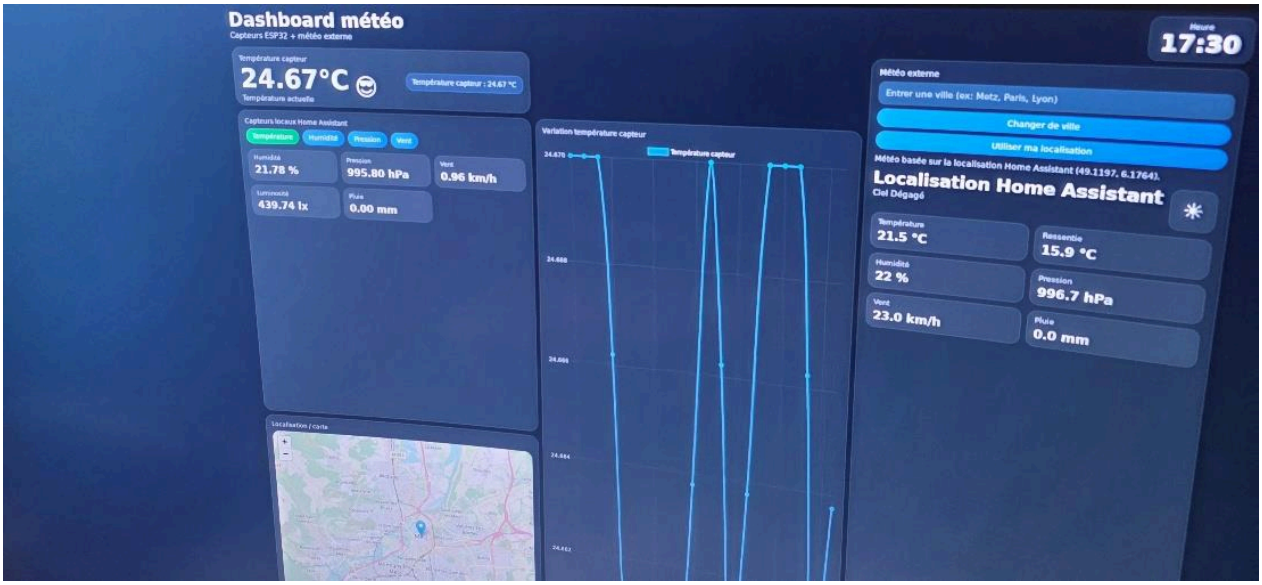


Figure 4 : Dashboard station météo finale

10. Conclusion

Ce TP m'a permis d'aborder concrètement plusieurs concepts importants : l'architecture REST, la communication entre un système embarqué (ESP32 + capteurs) et une interface utilisateur via une API, et

l'optimisation de code front-end pour une application mise à jour en continu. La séparation claire entre les groupes (ESP32, Home Assistant, HTML, maquette du site) a permis à chacun de se concentrer sur son domaine, tout en gardant une vue d'ensemble grâce au Gantt commun.

Pour aller plus loin, plusieurs améliorations seraient possibles dans une version 4.0 : remplacer le polling par l'API WebSocket de Home Assistant pour recevoir les changements d'état en temps réel, remplacer le `prompt()` du token par un vrai écran de connexion, transformer le dashboard en PWA (Progressive Web App) pour pouvoir l'installer sur un téléphone, ou encore mettre en place un proxy Node.js pour garder le token côté serveur.

11. Annexe- Codes complets

11.1 Code HTML optimisé (index.html)

```
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width,initial-scale=1" />
  <meta name="referrer" content="strict-origin-when-cross-origin" />
  <meta name="description" content="Dashboard météo connecté - Capteurs ESP32 + Home Assistant + Open-
Meteo" />
  <meta name="theme-color" content="#1d3557" />
  <title>Dashboard Météo Home Assistant</title>

  <!-- Préconnexion aux serveurs externes : connexions TCP/TLS ouvertes en avance -->
  <link rel="preconnect" href="https://unpkg.com" crossorigin />
  <link rel="preconnect" href="https://cdn.jsdelivr.net" crossorigin />
  <link rel="preconnect" href="https://tile.openstreetmap.org" />
  <link rel="preconnect" href="https://api.open-meteo.com" />
  <link rel="preconnect" href="https://geocoding-api.open-meteo.com" />

  <link rel="stylesheet" href="./style.css" />
  <link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css"
crossorigin="anonymous" />
</head>
<body>
  <main class="appShell" role="main">


    <!-- ============ BARRE DU HAUT : titre + horloge ============ -->
    <header class="topBar">
      <div>
        <h1>Dashboard météo</h1>
        <p class="subtitle">Capteurs ESP32 + météo externe</p>
      </div>

      <div class="clockBox" aria-label="Horloge">
        <div class="clockLabel">Heure</div>
        <div class="clock" id="clock" aria-live="off">--:--</div>
      </div>
    </header>

    <div class="dashboardGrid">

      <!-- ============ HÉRO : grosse température + emoji + statut ============ -->
      <section class="panel heroPanel" aria-label="Température principale">
        <div class="heroLeft">
          <div class="smallTitle">Température capteur</div>

          <div class="tempRow">
            <div class="tempBlock">
              <div class="temp" id="tempValue" aria-live="polite">--°C</div>
              <div class="tempLabel">Température actuelle</div>
            </div>
            <div id="emoji" class="emoji" aria-hidden="true">😊</div>
          </div>
        </div>

        <div class="heroRight">
          <div class="statusBadge" id="hint" role="status" aria-live="polite">Chargement…</div>
        </div>
      </section>

      <!-- ============ CAPTEURS LOCAUX (HA) ============ -->
      <section class="panel leftPanel" aria-label="Capteurs locaux">
```

```
        <div class="boxTitle">Capteurs locaux Home Assistant</div>

        <div class="btnCol" role="tablist" aria-label="Sélection du graphique">
          <button class="pill active" id="btnTemperature" role="tab" aria-
selected="true">Température</button>
          <button class="pill" id="btnHumidity" role="tab" aria-selected="false">Humidité</button>
          <button class="pill" id="btnPressure" role="tab" aria-selected="false">Pression</button>
          <button class="pill" id="btnWind" role="tab" aria-selected="false">Vent</button>
        </div>

        <div class="statsGrid">
          <div class="statCard"><span>Humidité</span><b id="humValue" aria-live="polite">N/A</b></div>
          <div class="statCard"><span>Pression</span><b id="presValue" aria-live="polite">N/A</b></div>

          <div class="statCard"><span>Vent</span><b id="windValue" aria-live="polite">N/A</b></div>
          <div class="statCard"><span>Luminosité</span><b id="lightValue" aria-live="polite">N/A</b>
</div>
          <div class="statCard"><span>Pluie</span><b id="rainValue" aria-live="polite">N/A</b></div>
        </div>
      </section>

      <!-- ============ MÉTÉO EXTERNE (Open-Meteo) ============ -->
      <section class="panel rightPanel" aria-label="Météo externe">
        <div class="boxTitle">Météo externe</div>

        <div class="weatherConfig">
          <!-- label visuellement caché pour l'accessibilité -->
          <label for="cityInput" class="visuallyHidden">Ville à rechercher</label>
          <input type="text" id="cityInput" placeholder="Entrer une ville (ex: Metz, Paris, Lyon)"
autocomplete="off" />
          <button class="pill" id="btnSearchCity">Changer de ville</button>
          <button class="pill" id="btnUseHomeLocation">Utiliser ma localisation</button>
        </div>

        <div class="weatherStatus" id="weatherStatus" role="status" aria-live="polite">
          La météo se charge depuis la localisation Home Assistant.
        </div>

        <div class="panelScroll weatherContent">
          <div class="weatherHeader">
            <div>
              <div class="weatherCity" id="extCity">--</div>
              <div class="weatherDesc" id="extDesc">--</div>
            </div>
            <div id="extIcon" class="weatherIconEmoji" aria-hidden="true">☁️</div>
          </div>

          <div class="statsGrid weatherStatsGrid">
            <div class="statCard"><span>Température</span><b id="extTemp">N/A</b></div>
            <div class="statCard"><span>Ressentie</span><b id="extFeels">N/A</b></div>
            <div class="statCard"><span>Humidité</span><b id="extHum">N/A</b></div>
            <div class="statCard"><span>Pression</span><b id="extPres">N/A</b></div>
            <div class="statCard"><span>Vent</span><b id="extWind">N/A</b></div>
            <div class="statCard"><span>Pluie</span><b id="extRain">N/A</b></div>
          </div>
        </div>
      </section>

      <!-- ============ CARTE LEAFLET ============ -->

      <section class="panel mapPanel" aria-label="Localisation">
        <div class="boxTitle">Localisation / carte</div>
        <div class="map" id="map"></div>
      </section>

      <!-- ============ GRAPHIQUE Chart.js ============ -->
      <section class="panel chartPanel" aria-label="Graphique des capteurs">
        <div class="boxTitle" id="chartTitle">Variation température capteur</div>
        <div class="chartWrap">
          <canvas id="chart"></canvas>
```

```
        </div>
      </section>

    </div>
  </main>

  <!-- defer = téléchargé en parallèle, exécuté après parsing du DOM -->
  <script defer src="https://cdn.jsdelivr.net/npm/chart.js@4.4.1/dist/chart.umd.min.js"></script>
  <script defer src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js" crossorigin="anonymous"></script>
  <script defer src="./app.js"></script>
</body>
</html>
```

11.2 Code CSS optimisé (style.css)

```
/* ============================================================
   1) RESET GLOBAL
   On remet tout à zéro pour partir d'une base propre
============================================================ */
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

/* Classe utilitaire pour cacher visuellement un label tout en
   le gardant accessible pour les lecteurs d'écran */
.visuallyHidden {
  position: absolute;
  width: 1px; height: 1px;
  padding: 0; margin: -1px;
  overflow: hidden; clip: rect(0,0,0,0);
  white-space: nowrap; border: 0;
}

/* ============================================================
   2) VARIABLES GLOBALES (couleurs + tailles + espacements)
   Tout est centralisé ici pour pouvoir changer le thème
   en touchant un seul endroit du fichier
============================================================ */
:root {
  /* Fond dégradé */
  --bg1: #1d3557;

  --bg2: #0f172a;
  --bg3: #08101f;

  /* Panneaux et cartes (effet glassmorphism) */
  --panel: rgba(255,255,255,0.08);
  --panel-border: rgba(255,255,255,0.12);
  --card: rgba(255,255,255,0.07);
  --card-border: rgba(255,255,255,0.10);

  --text-soft: #cbd5e1;
  --input-bg: #314764;

  /* Espacements et arrondis */
  --gap: 10px;
  --radius: 20px;
  --outer-pad: 10px;

  /* Hauteurs principales */
  --row-top-height: 120px;
  --row-middle-min: 315px;
  --row-bottom-min: 235px;

  /* Tailles fluides (clamp = min, ideal, max) qui s'adaptent
     à la taille de l'écran sans avoir besoin de breakpoints */
```

```
  --temp-main-size: clamp(34px, 3vw, 54px);
  --temp-label-size: clamp(12px, 0.85vw, 15px);
  --emoji-size: clamp(32px, 2.5vw, 48px);
  --weather-city-size: clamp(18px, 1.6vw, 30px);
  --weather-desc-size: clamp(12px, 0.8vw, 15px);
  --stat-label-size: clamp(11px, 0.72vw, 13px);
  --stat-value-size: clamp(14px, 1vw, 22px);
  --button-text-size: clamp(12px, 0.82vw, 14px);
  --weather-icon-box: clamp(48px, 3.6vw, 68px);
  --weather-icon-size: clamp(24px, 1.7vw, 34px);
}

/* ============================================================
   3) BASE HTML / BODY
============================================================ */
html, body {
  width: 100%;
  height: 100%;
}

body {
  font-family: Arial, sans-serif;
  /* Dégradé radial qui part du coin haut gauche */
  background: radial-gradient(circle at top left, var(--bg1) 0%, var(--bg2) 35%, var(--bg3) 100%);
  color: #ffffff;
  min-height: 100dvh;       /* 100dvh = mieux que 100vh sur mobile (gère la barre du navigateur) */
  padding: var(--outer-pad);
  overflow: hidden;
}

/* Accessibilité : si l'utilisateur a coché "réduire les animations",
   on coupe toutes les transitions */
@media (prefers-reduced-motion: reduce) {

  * { transition: none !important; animation: none !important; }
}

/* ============================================================
   4) CONTENEUR PRINCIPAL
============================================================ */
.appShell {
  width: 100%;
  max-width: 1900px;
  margin: 0 auto;
  height: calc(100dvh - (var(--outer-pad) * 2));
  display: grid;
  grid-template-rows: auto 1fr;
  gap: var(--gap);
  min-height: 0;
}

/* ============================================================
   5) BARRE DU HAUT (titre + horloge)
============================================================ */
.topBar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 12px;
}

.topBar h1 {
  font-size: clamp(22px, 2vw, 38px);
  margin-bottom: 2px;
  line-height: 1.05;
}

.subtitle {
  color: var(--text-soft);
  font-size: clamp(12px, 0.9vw, 16px);
```

```
}

.clockBox {
  background: var(--panel);
  border: 1px solid var(--panel-border);
  backdrop-filter: blur(8px);
  border-radius: 16px;
  padding: 8px 16px;
  min-width: 110px;
  text-align: center;
  flex-shrink: 0;
}

.clockLabel {
  font-size: 11px;
  color: var(--text-soft);
  margin-bottom: 2px;
}

.clock {
  font-size: clamp(24px, 1.8vw, 36px);
  font-weight: bold;
  line-height: 1;

  /* Optimisation rendu : le navigateur sait que le contenu change souvent */
  will-change: contents;
}

/* ============================================================
   6) GRILLE PRINCIPALE DU DASHBOARD
   2 colonnes × 3 lignes
============================================================ */
.dashboardGrid {
  min-height: 0;
  display: grid;
  grid-template-columns: 1fr 1fr;
  grid-template-rows:
    var(--row-top-height)
    minmax(var(--row-middle-min), 1.3fr)
    minmax(var(--row-bottom-min), 1fr);
  gap: var(--gap);
}

/* ============================================================
   7) STYLE COMMUN DES PANNEAUX (glassmorphism)
============================================================ */
.panel {
  background: var(--panel);
  border: 1px solid var(--panel-border);
  border-radius: var(--radius);
  padding: 14px;
  backdrop-filter: blur(10px);
  box-shadow: 0 10px 30px rgba(0,0,0,0.20);
  min-width: 0;
  min-height: 0;
  overflow: hidden;
  /* Isole le repaint du panneau du reste de la page : grosse optim perf
*/
  contain: layout style;
}

/* ============================================================
   8) POSITIONNEMENT DES BLOCS DANS LA GRILLE
============================================================ */
/* Bandeau du haut : occupe les 2 colonnes */
.heroPanel {
  grid-column: 1 / 3;
  grid-row: 1;
  display: grid;
  grid-template-columns: 1fr auto;
```

```
  align-items: center;
  gap: 12px;
}

/* Capteurs locaux : colonne gauche, ligne du milieu */
.leftPanel { grid-column: 1; grid-row: 2; }

/* Météo externe : colonne droite, ligne du milieu */
.rightPanel {
  grid-column: 2;
  grid-row: 2;
  display: grid;

  grid-template-rows: auto auto auto 1fr;
  min-height: 0;
}

/* Carte : colonne gauche, ligne du bas */
.mapPanel {
  grid-column: 1;
  grid-row: 3;
  display: flex;
  flex-direction: column;
  min-height: 0;
}

/* Graphique : colonne droite, ligne du bas */
.chartPanel {
  grid-column: 2;
  grid-row: 3;
  display: flex;
  flex-direction: column;
  min-height: 0;
}

/* ============================================================
   9) TITRES DE BLOCS
============================================================ */
.heroLeft, .heroRight { min-width: 0; }

.smallTitle, .boxTitle {
  color: var(--text-soft);
  font-size: clamp(12px, 0.8vw, 14px);
  margin-bottom: 8px;
  line-height: 1.2;
}

/* ============================================================
   10) BLOC TEMPÉRATURE PRINCIPALE
============================================================ */
.tempRow {
  display: flex;
  align-items: center;
  gap: 12px;
  flex-wrap: wrap;
}

.tempBlock { min-width: 0; }

.temp {
  font-size: var(--temp-main-size);
  font-weight: bold;
  line-height: 0.95;
  word-break: break-word;
}

.tempLabel {
  margin-top: 6px;
  color: var(--text-soft);
  font-size: var(--temp-label-size);
```

```
  line-height: 1.2;
}

.emoji {
  font-size: var(--emoji-size);
  line-height: 1;
  flex-shrink: 0;
}

/* ========================================================
   11) BADGE DE STATUT
======================================================== */
.statusBadge {
  justify-self: end;
  background: rgba(59,130,246,0.18);
  border: 1px solid rgba(96,165,250,0.35);
  color: #dbeafe;
  padding: 8px 12px;
  border-radius: 14px;
  max-width: 100%;
  word-break: break-word;
  font-size: clamp(11px, 0.8vw, 14px);
  line-height: 1.25;
}

/* ========================================================
   12) BOUTONS PILULE
======================================================== */
.btnCol {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
  margin-bottom: 12px;
}

.pill {
  border: none;
  background: linear-gradient(135deg, #3b82f6, #2563eb);
  color: white;
  padding: 8px 14px;
  border-radius: 999px;
  cursor: pointer;
  font-size: var(--button-text-size);
  transition: transform .2s ease, opacity .2s ease;
  white-space: nowrap;
}

.pill:hover { transform: translateY(-1px); opacity: .95; }

.pill.active {
  background: linear-gradient(135deg, #22c55e, #16a34a);
  box-shadow: 0 0 0 2px rgba(255,255,255,0.15) inset;
}

.pill:focus-visible {
  outline: 2px solid #60a5fa;
  outline-offset: 2px;
}

/* ========================================================
   13) GRILLES DE CARTES (capteurs)

======================================================== */
.statsGrid {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 10px;
}

.weatherStatsGrid {
```

```
  grid-template-columns: repeat(2, minmax(0, 1fr));
}

.statCard {
  background: var(--card);
  border: 1px solid var(--card-border);
  border-radius: 16px;
  padding: 10px 12px;
  min-width: 0;
  min-height: 0;
}

.statCard span {
  display: block;
  color: var(--text-soft);
  font-size: var(--stat-label-size);
  margin-bottom: 6px;
  line-height: 1.15;
}

.statCard b {
  display: block;
  font-size: var(--stat-value-size);
  line-height: 1.15;
  word-break: break-word;
}

/* ============================================================
   14) FORMULAIRE MÉTÉO EXTERNE
============================================================ */
.weatherConfig {
  display: grid;
  gap: 8px;
  margin-bottom: 10px;
}

.weatherConfig input {
  width: 100%;
  min-width: 0;
  padding: 10px 12px;
  border-radius: 12px;
  border: 1px solid rgba(255,255,255,0.08);
  background: var(--input-bg);
  color: white;
  font-size: clamp(12px, 0.82vw, 15px);
  outline: none;
}

.weatherConfig input::placeholder { color: #b7c6d8; }

.weatherConfig input:focus-visible {

  border-color: #60a5fa;
  box-shadow: 0 0 0 2px rgba(96,165,250,0.3);
}

.weatherStatus {
  margin-bottom: 10px;
  color: var(--text-soft);
  font-size: clamp(11px, 0.75vw, 14px);
  word-break: break-word;
  line-height: 1.25;
}

/* ============================================================
   15) ZONE SCROLLABLE INTERNE
============================================================ */
.panelScroll {
  min-height: 0;
  overflow-y: auto;
```

```
  overflow-x: hidden;
  padding-right: 4px;
}

.weatherContent { min-height: 0; }

/* ============================================================
   16) EN-TÊTE DE LA MÉTÉO EXTERNE
============================================================ */
.weatherHeader {
  display: flex;
  justify-content: space-between;
  align-items: flex-start;
  gap: 12px;
  margin-bottom: 12px;
}

.weatherHeader > div:first-child { min-width: 0; flex: 1; }

.weatherCity {
  font-size: var(--weather-city-size);
  font-weight: bold;
  line-height: 1.05;
  word-break: break-word;
}

.weatherDesc {
  color: var(--text-soft);
  text-transform: capitalize;
  margin-top: 4px;
  font-size: var(--weather-desc-size);
  line-height: 1.2;
  word-break: break-word;
}

.weatherIconEmoji {
  width: var(--weather-icon-box);
  height: var(--weather-icon-box);
  display: grid;
  place-items: center;
  font-size: var(--weather-icon-size);

  line-height: 1;
  border-radius: 16px;
  background: rgba(255,255,255,0.07);
  border: 1px solid rgba(255,255,255,0.10);
  flex-shrink: 0;
}

/* ============================================================
   17) CARTE ET GRAPHIQUE
============================================================ */
.map {
  width: 100%;
  flex: 1;
  min-height: 0;
  border-radius: 18px;
  overflow: hidden;
}

.chartWrap {
  width: 100%;
  flex: 1;
  min-height: 0;
  position: relative;
}

canvas {
  width: 100% !important;
  height: 100% !important;
```

```
}

/* ============================================================
   18) SCROLLBAR DISCRÈTE
============================================================ */
.panelScroll::-webkit-scrollbar { width: 8px; }
.panelScroll::-webkit-scrollbar-track {
  background: rgba(255,255,255,0.05);
  border-radius: 999px;
}
.panelScroll::-webkit-scrollbar-thumb {
  background: rgba(255,255,255,0.18);
  border-radius: 999px;
}

/* ============================================================
   19) RESPONSIVE — Écrans bas (laptops)
============================================================ */
@media (max-height: 930px) {
  :root {
    --row-top-height: 110px;
    --row-middle-min: 290px;
    --row-bottom-min: 215px;
    --temp-main-size: clamp(30px, 2.6vw, 46px);
    --weather-city-size: clamp(16px, 1.35vw, 26px);
    --stat-value-size: clamp(13px, 0.92vw, 18px);
  }
}

/* ============================================================
   20) RESPONSIVE — Écrans moyens

============================================================ */
@media (max-width: 1450px) {
  .statsGrid, .weatherStatsGrid {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }
}

/* ============================================================
   21) RESPONSIVE — Tablette / petit écran
============================================================ */
@media (max-width: 1100px) {
  body { overflow: auto; }

  .appShell {
    height: auto;
    min-height: calc(100dvh - (var(--outer-pad) * 2));
  }

  .dashboardGrid {
    grid-template-columns: 1fr;
    grid-template-rows: auto;
  }

  .heroPanel, .leftPanel, .rightPanel, .mapPanel, .chartPanel {
    grid-column: auto;
    grid-row: auto;
  }

  .heroPanel {
    display: flex;
    flex-direction: column;
    align-items: flex-start;
  }

  .panel { overflow: visible; }
  .panelScroll, .weatherContent { overflow: visible; }
  .statsGrid, .weatherStatsGrid {
    grid-template-columns: repeat(2, minmax(0, 1fr));
```

```
  }
  .map, .chartWrap {
    min-height: 260px;
    height: 260px;
  }
}

/* ============================================================
   22) RESPONSIVE — Mobile
============================================================ */
@media (max-width: 650px) {
  .topBar {
    flex-direction: column;
    align-items: stretch;
  }
  .clockBox { width: 100%; }
  .btnCol { display: grid; grid-template-columns: 1fr 1fr; }
  .pill { width: 100%; text-align: center; }
  .statsGrid, .weatherStatsGrid { grid-template-columns: 1fr; }
}
```

11.3 Code JavaScript optimisé (app.js)

```
// ========================================================================
// app.js — Dashboard Météo Home Assistant + Open-Meteo
// Auteur : BELKACEM Mohamed Amine
// ========================================================================

// ==================== CONFIG ====================
// Token chargé depuis sessionStorage. Si pas trouvé, on demande à l'user.
// JAMAIS en clair dans le code (risque sécu si on commit sur GitHub public)
const TOKEN = (() => {
  let t = sessionStorage.getItem("ha_token");
  if (!t) {
    t = prompt("Colle ton Long-Lived Access Token Home Assistant :") || "";
    if (t) sessionStorage.setItem("ha_token", t);
  }
  return t;
})();

const CONFIG = Object.freeze({
  homeAssistant: {
    location: "zone.home",
    entities: {
      temperature: "sensor.espbonbon_temperature_2",
      humidity:    "sensor.espbonbon_humidite_2",
      pressure:    "sensor.espbonbon_pression_2",
      wind:        "sensor.esp32_weather_vent",
      light:       "sensor.espbonbon_luminosite_2",
      rain:        "sensor.espbonbon_pluviometrie",
    },
  },
  refreshMs: 5000,             // polling capteurs locaux
  weatherRefreshMs: 600000,    // météo externe : toutes les 10 min
  maxChartPoints: 40,          // 40 points au lieu de 20 = meilleur historique visible
  fetchTimeoutMs: 8000,
  fetchRetries: 2,
});

// Headers HTTP créés UNE seule fois et gelés
const AUTH_HEADERS = Object.freeze({
  "Authorization": `Bearer ${TOKEN}`,
  "Content-Type":  "application/json",
});

// ==================== CACHE DOM ====================
const el = Object.freeze({
  temp:  document.getElementById("tempValue"),
```

```
  hum:   document.getElementById("humValue"),
  pres:  document.getElementById("presValue"),
  wind:  document.getElementById("windValue"),
  light: document.getElementById("lightValue"),

  rain:  document.getElementById("rainValue"),
  clock: document.getElementById("clock"),
  hint:  document.getElementById("hint"),
  emoji: document.getElementById("emoji"),
  chartTitle: document.getElementById("chartTitle"),
  btnTemperature: document.getElementById("btnTemperature"),
  btnHumidity:    document.getElementById("btnHumidity"),
  btnPressure:    document.getElementById("btnPressure"),
  btnWind:        document.getElementById("btnWind"),
  cityInput:      document.getElementById("cityInput"),
  btnSearchCity:  document.getElementById("btnSearchCity"),
  btnUseHomeLocation: document.getElementById("btnUseHomeLocation"),
  weatherStatus:  document.getElementById("weatherStatus"),
  extCity: document.getElementById("extCity"),
  extDesc: document.getElementById("extDesc"),
  extIcon: document.getElementById("extIcon"),
  extTemp: document.getElementById("extTemp"),
  extFeels: document.getElementById("extFeels"),
  extHum: document.getElementById("extHum"),
  extPres: document.getElementById("extPres"),
  extWind: document.getElementById("extWind"),
  extRain: document.getElementById("extRain"),
});

const CHART_SERIES = Object.freeze({
  temperature: { label: "Température capteur", title: "Variation température capteur", entityKey:
"temperature", fallbackUnit: "°C" },
  humidity:    { label: "Humidité capteur",    title: "Variation humidité capteur",    entityKey:
"humidity",    fallbackUnit: "%" },
  pressure:    { label: "Pression capteur",    title: "Variation pression capteur",    entityKey:
"pressure",    fallbackUnit: "hPa" },
  wind:        { label: "Vent capteur",        title: "Variation vent capteur",        entityKey:
"wind",        fallbackUnit: "km/h" },
});

let currentChartKey = "temperature";
let chart = null;
let mapInstance = null;
let marker = null;
let currentExternalCoords = null;
let currentExternalPlaceName = null;
let refreshTimer = null;
let weatherTimer = null;

// Cache du géocoding (clé = ville en minuscules, valeur = coordonnées).
// Comme ça si l'user retape "Paris" 5 fois, on tape l'API qu'UNE seule fois.
const geocodeCache = new Map();

// Historique du chart — persisté dans sessionStorage pour survivre aux refresh
const historyStore = (() => {
  const saved = sessionStorage.getItem("history_store");
  if (saved) {
    try { return JSON.parse(saved); } catch { /* ignore */ }
  }
  return {
    temperature: { labels: [], data: [] },
    humidity:    { labels: [], data: [] },

    pressure:    { labels: [], data: [] },
    wind:        { labels: [], data: [] },
  };
})();

function saveHistory() {
  sessionStorage.setItem("history_store", JSON.stringify(historyStore));
```

```
}

// ==================== EMOJIS ====================
function getTemperatureEmoji(value) {
  if (value >= 30) return "🥵";
  if (value >= 18) return "😎";
  if (value >= 14) return "😊";
  if (value >= 4)  return "🥶";
  return "❄️";
}

function getWeatherEmoji(code) {
  if (code === 0) return "☀️";
  if (code === 1) return "🌤️";
  if (code === 2) return "⛅";
  if (code === 3) return "☁️";
  if ([45, 48].includes(code)) return "🌫️";
  if ([51,53,55,56,57,61,63,65,80,81,82].includes(code)) return "🌧️";
  if ([66, 67].includes(code)) return "🌨️";
  if ([71,73,75,77,85,86].includes(code)) return "❄️";
  if ([95, 96, 99].includes(code)) return "⛈️";
  return "🌤️";
}

const WEATHER_DESCRIPTIONS = Object.freeze({
  0:"Ciel dégagé", 1:"Plutôt dégagé", 2:"Partiellement nuageux", 3:"Couvert",
  45:"Brouillard", 48:"Brouillard givrant",
  51:"Bruine légère", 53:"Bruine modérée", 55:"Bruine dense",
  56:"Bruine verglaçante légère", 57:"Bruine verglaçante dense",
  61:"Pluie faible", 63:"Pluie modérée", 65:"Pluie forte",
  66:"Pluie verglaçante légère", 67:"Pluie verglaçante forte",
  71:"Neige faible", 73:"Neige modérée", 75:"Neige forte", 77:"Grains de neige",
  80:"Averses faibles", 81:"Averses modérées", 82:"Averses fortes",
  85:"Averses de neige faibles", 86:"Averses de neige fortes",
  95:"Orage", 96:"Orage avec grêle légère", 99:"Orage avec forte grêle",
});

function weatherDescriptionFr(code) {
  return WEATHER_DESCRIPTIONS[code] || "Conditions météo inconnues";
}

// ==================== HORLOGE (alignée sur la seconde) ====================
function startClock() {
  const tick = () => {
    const d = new Date();

    el.clock.textContent =
 `${String(d.getHours()).padStart(2,"0")}:${String(d.getMinutes()).padStart(2,"0")}`;
    // Aligné pile sur la prochaine seconde → pas de dérive
    setTimeout(tick, 1000 - (Date.now() % 1000));
  };
  tick();
}

// ==================== FETCH AVEC TIMEOUT + RETRY ====================
async function safeFetch(url, { method = "GET", headers, body, retries = CONFIG.fetchRetries, timeoutMs
= CONFIG.fetchTimeoutMs } = {}) {
  for (let attempt = 0; attempt <= retries; attempt++) {
    const controller = new AbortController();
    const timer = setTimeout(() => controller.abort(), timeoutMs);
    try {
      const res = await fetch(url, { method, headers, body, signal: controller.signal });
      clearTimeout(timer);
      if (!res.ok) throw new Error(`HTTP ${res.status} sur ${url}`);
      return await res.json();  // await : une erreur de parsing JSON passe aussi par le retry
    } catch (err) {
      clearTimeout(timer);
      if (attempt === retries) throw err;
      await new Promise(r => setTimeout(r, 500 * 2 ** attempt));  // backoff : 500 ms puis 1 s (retries
```

```
= 2)
    }
  }
}

const haGet = (path) => safeFetch(path, { headers: AUTH_HEADERS });

// ==================== FORMATAGE ====================
function formatValue(stateObj, fallback = "N/A") {
  if (!stateObj) return fallback;
  if (stateObj.state === "unknown" || stateObj.state === "unavailable") return fallback;
  const raw = Number(stateObj.state);
  const unit = stateObj.attributes?.unit_of_measurement || "";
  if (Number.isFinite(raw)) return unit ? `${raw.toFixed(2)} ${unit}` : raw.toFixed(2);
  return unit ? `${stateObj.state} ${unit}` : stateObj.state;
}

function formatRainValue(stateObj) {
  if (!stateObj) return "N/A";
  if (stateObj.state === "unknown" || stateObj.state === "unavailable") return "N/A";
  const state = String(stateObj.state).toLowerCase();
  const deviceClass = stateObj.attributes?.device_class || "";
  if (deviceClass === "moisture" || ["on","wet","rain"].includes(state)) return "Oui";
  if (["off","dry","clear"].includes(state)) return "Non";
  return formatValue(stateObj);
}

// ==================== UI ====================
function updateUI(states) {

  const ent = CONFIG.homeAssistant.entities;
  const t = states.get(ent.temperature);

  if (t && t.state !== "unknown" && t.state !== "unavailable") {
    const value = Number(t.state);
    const unit = t.attributes?.unit_of_measurement || "°C";
    if (Number.isFinite(value)) {
      el.temp.textContent = `${value.toFixed(2)}${unit}`;
      el.emoji.textContent = getTemperatureEmoji(value);
    } else {
      el.temp.textContent = `--${unit}`;
      el.emoji.textContent = " ❓ ";
    }
  } else {
    el.temp.textContent = "--°C";
    el.emoji.textContent = " ❓ ";
  }

  el.hum.textContent   = formatValue(states.get(ent.humidity));
  el.pres.textContent  = formatValue(states.get(ent.pressure));
  el.wind.textContent  = formatValue(states.get(ent.wind));
  el.light.textContent = formatValue(states.get(ent.light));
  el.rain.textContent  = formatRainValue(states.get(ent.rain));
}

async function loadAllStates() {
  const list = await haGet("/api/states");
  // Map au lieu de {} : pas de clés héritées du prototype, lookup O(1) en moyenne
  const map = new Map();
  for (const s of list) map.set(s.entity_id, s);
  return map;
}

// ==================== CARTE LEAFLET ====================
function initMap(lat, lon) {
  mapInstance = L.map("map", { zoomControl: true }).setView([lat, lon], 12);
  L.tileLayer("https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png", {
    maxZoom: 19,
    attribution: "&copy; OpenStreetMap contributors",
    referrerPolicy: "origin",
```

```
  }).addTo(mapInstance);
  marker = L.marker([lat, lon]).addTo(mapInstance);
}

function updateMap(lat, lon) {
  if (!mapInstance) return initMap(lat, lon);
  marker.setLatLng([lat, lon]);
  mapInstance.setView([lat, lon], mapInstance.getZoom());
}

function getHomeCoordinates(states) {
  const loc = states.get(CONFIG.homeAssistant.location);
  const lat = loc?.attributes?.latitude;
  const lon = loc?.attributes?.longitude;
  return (typeof lat === "number" && typeof lon === "number") ? { lat, lon } : null;
}

function updateLocation(states) {
  const coords = getHomeCoordinates(states);
  if (coords) updateMap(coords.lat, coords.lon);
}

// ==================== GRAPHIQUE Chart.js ====================
function initChart() {
  const ctx = document.getElementById("chart");
  chart = new Chart(ctx, {
    type: "line",
    data: {
      labels: [],
      datasets: [{
        label: CHART_SERIES[currentChartKey].label,
        data: [],
        tension: 0.35,
        pointRadius: 2,
        pointHoverRadius: 5,
        borderWidth: 2.5,
        fill: false,
      }],
    },
    options: {
      responsive: true,
      maintainAspectRatio: false,
      animation: false,           // ⚡ gain de perf énorme sur les updates fréquents
      plugins: { legend: { labels: { color: "#ffffff" } } },
      scales: {
        x: { ticks: { color:"#cbd5e1", maxTicksLimit: 8 }, grid: { color:"rgba(255,255,255,0.08)" } },
        y: { ticks: { color:"#cbd5e1" }, grid: { color:"rgba(255,255,255,0.08)" }, beginAtZero: false
},
      },
    },
  });
}

function getCurrentTimeLabel() {
  const d = new Date();
  return
`${String(d.getHours()).padStart(2,"0")}:${String(d.getMinutes()).padStart(2,"0")}:${String(d.getSecond
s()).padStart(2,"0")}`;
}

function pushHistory(chartKey, value) {
  if (!Number.isFinite(value)) return;
  const store = historyStore[chartKey];
  store.labels.push(getCurrentTimeLabel());
  store.data.push(Number(value.toFixed(2)));
  if (store.labels.length > CONFIG.maxChartPoints) {
    store.labels.shift();
    store.data.shift();
  }
}
```

```
function renderCurrentChart() {

  if (!chart) initChart();
  const series = CHART_SERIES[currentChartKey];
  const store = historyStore[currentChartKey];
  chart.data.labels = store.labels;
  chart.data.datasets[0].data = store.data;
  chart.data.datasets[0].label = series.label;
  chart.update();
  el.chartTitle.textContent = series.title;
}

function setActiveButton(chartKey) {
  const buttons = {
    temperature: el.btnTemperature,
    humidity: el.btnHumidity,
    pressure: el.btnPressure,
    wind: el.btnWind,
  };
  for (const [key, btn] of Object.entries(buttons)) {
    if (!btn) continue;
    const isActive = key === chartKey;
    btn.classList.toggle("active", isActive);
    btn.setAttribute("aria-selected", isActive);
  }
}

function switchChart(chartKey) {
  currentChartKey = chartKey;
  setActiveButton(chartKey);
  renderCurrentChart();
  el.hint.textContent = `Graphique affiché : ${CHART_SERIES[chartKey].label}`;
}

function updateAllHistories(states) {
  const ent = CONFIG.homeAssistant.entities;
  const keys = ["temperature", "humidity", "pressure", "wind"];
  for (const key of keys) {
    const s = states.get(ent[key]);
    if (!s || s.state === "unknown" || s.state === "unavailable") continue;
    const v = Number(s.state);
    if (Number.isFinite(v)) pushHistory(key, v);
  }
  saveHistory();    // persistance dans sessionStorage
}

// ==================== GEOCODING + MÉTÉO EXTERNE ====================
async function geocodeCity(city) {
  const key = city.toLowerCase().trim();
  // Si déjà cherchée, on prend en cache au lieu de retaper l'API
  if (geocodeCache.has(key)) return geocodeCache.get(key);

  const url = `https://geocoding-api.open-meteo.com/v1/search?
name=${encodeURIComponent(city)}&count=1&language=fr&format=json`;
  const data = await safeFetch(url);
  if (!data.results?.length) throw new Error("Ville introuvable.");

  const place = data.results[0];

  const result = {
    lat: place.latitude,
    lon: place.longitude,
    name: place.name,
    country: place.country || "",
  };
  geocodeCache.set(key, result);   // on cache pour les prochains appels
  return result;
}
```

```
async function fetchMeteoFranceByCoords(lat, lon) {
  const url = `https://api.open-meteo.com/v1/meteofrance?latitude=${lat}&longitude=${lon}`
            + `&current=temperature_2m,relative_humidity_2m,apparent_temperature,surface_pressure,`
            + `wind_speed_10m,weather_code,precipitation,rain&timezone=auto`;
  return safeFetch(url);
}

function updateExternalWeatherUI(data, placeLabel = "--") {
  const c = data.current || {};
  el.extCity.textContent = placeLabel;
  el.extDesc.textContent = weatherDescriptionFr(c.weather_code);

  el.extTemp.textContent  = Number.isFinite(c.temperature_2m)       ? `${c.temperature_2m.toFixed(1)}
°C` : "N/A";
  el.extFeels.textContent = Number.isFinite(c.apparent_temperature) ?
`${c.apparent_temperature.toFixed(1)} °C` : "N/A";
  el.extHum.textContent   = Number.isFinite(c.relative_humidity_2m) ? `${c.relative_humidity_2m} %` :
"N/A";
  el.extPres.textContent  = Number.isFinite(c.surface_pressure)     ? `${c.surface_pressure.toFixed(1)}
hPa` : "N/A";
  el.extWind.textContent  = Number.isFinite(c.wind_speed_10m)       ? `${c.wind_speed_10m.toFixed(1)}
km/h` : "N/A";

  if (Number.isFinite(c.rain) || Number.isFinite(c.precipitation)) {
    const r = Number.isFinite(c.rain) ? c.rain : c.precipitation;
    el.extRain.textContent = `${r.toFixed(1)} mm`;
  } else {
    el.extRain.textContent = "0.0 mm";
  }

  el.extIcon.textContent = getWeatherEmoji(c.weather_code);
}

async function refreshExternalWeatherFromHome(states) {
  const coords = getHomeCoordinates(states);
  if (!coords) {
    el.weatherStatus.textContent = "Impossible de lire la localisation Home Assistant.";
    return;
  }
  currentExternalCoords = coords;
  currentExternalPlaceName = "Localisation Home Assistant";
  el.weatherStatus.textContent = "Chargement de la météo locale…";
  const data = await fetchMeteoFranceByCoords(coords.lat, coords.lon);
  updateExternalWeatherUI(data, currentExternalPlaceName);

  el.weatherStatus.textContent = `Météo basée sur HA (${coords.lat.toFixed(4)},
${coords.lon.toFixed(4)}).`;
}

async function refreshExternalWeatherFromCity(city) {
  el.weatherStatus.textContent = `Recherche de "${city}"…`;
  const place = await geocodeCity(city);
  currentExternalCoords = { lat: place.lat, lon: place.lon };
  currentExternalPlaceName = place.country ? `${place.name}, ${place.country}` : place.name;
  const data = await fetchMeteoFranceByCoords(place.lat, place.lon);
  updateExternalWeatherUI(data, currentExternalPlaceName);
  updateMap(place.lat, place.lon);
  el.weatherStatus.textContent = `Météo chargée pour ${currentExternalPlaceName}.`;
}

// ==================== DEBOUNCE ====================
function debounce(fn, wait = 250) {
  let id;
  return (...args) => { clearTimeout(id); id = setTimeout(() => fn(...args), wait); };
}

// ==================== BOUCLE PRINCIPALE ====================
async function refreshData() {
  try {
    const states = await loadAllStates();
```

```
    updateUI(states);
    updateLocation(states);
    updateAllHistories(states);
    renderCurrentChart();

    const key = CHART_SERIES[currentChartKey].entityKey;
    const currentState = states.get(CONFIG.homeAssistant.entities[key]);

    if (currentState && currentState.state !== "unknown" &&currentState.state !== "unavailable") {
      const v = Number(currentState.state);
      if (Number.isFinite(v)) {
        const unit = currentState.attributes?.unit_of_measurement ||
CHART_SERIES[currentChartKey].fallbackUnit;
        el.hint.textContent = `${CHART_SERIES[currentChartKey].label} : ${v.toFixed(2)} ${unit}`;
      } else {
        el.hint.textContent = `${CHART_SERIES[currentChartKey].label} non numérique.`;
      }
    } else {
      el.hint.textContent = `${CHART_SERIES[currentChartKey].label} indisponible.`;
    }

    if (!currentExternalCoords) await refreshExternalWeatherFromHome(states);
  } catch (e) {
    console.error(e);
    el.hint.textContent = `Erreur Home Assistant : ${e.message}`;
  }

}

async function periodicRefreshExternalWeather() {
  if (!currentExternalCoords) return;
  try {
    const data = await fetchMeteoFranceByCoords(currentExternalCoords.lat, currentExternalCoords.lon);
    updateExternalWeatherUI(data, currentExternalPlaceName || "Météo externe");
  } catch (e) {
    console.error(e);
    el.weatherStatus.textContent = `Erreur météo : ${e.message}`;
  }
}

// ==================== ÉVÉNEMENTS ====================
function setupButtons() {
  el.btnTemperature.onclick = () => switchChart("temperature");
  el.btnHumidity.onclick    = () => switchChart("humidity");
  el.btnPressure.onclick    = () => switchChart("pressure");
  el.btnWind.onclick        = () => switchChart("wind");
  setActiveButton(currentChartKey);

  // Recherche débounced : si l'user tape vite, on attend qu'il se calme
  const doCitySearch = debounce(async () => {
    const city = el.cityInput.value.trim();
    if (!city) { el.weatherStatus.textContent = "Entre une ville d'abord."; return; }
    try { await refreshExternalWeatherFromCity(city); }
    catch (e) { console.error(e); el.weatherStatus.textContent = e.message; }
  }, 200);

  el.btnSearchCity.onclick = doCitySearch;
  el.cityInput.addEventListener("keydown", (event) => {
    if (event.key === "Enter") doCitySearch();
  });

  el.btnUseHomeLocation.onclick = async () => {
    try {
      const states = await loadAllStates();
      await refreshExternalWeatherFromHome(states);
    } catch (e) { console.error(e); el.weatherStatus.textContent = e.message; }
  };
}

// ==================== NETTOYAGE ====================
```

```
// Quand l'user quitte la page, on ferme tout proprement
window.addEventListener("beforeunload", () => {
  if (refreshTimer) clearInterval(refreshTimer);
  if (weatherTimer) clearInterval(weatherTimer);
  saveHistory();
});

// ==================== DÉMARRAGE ====================
(function main() {
  if (!TOKEN) {

    el.hint.textContent = "Token manquant. Rafraîchis la page pour le saisir.";
    return;
  }

  startClock();
  setupButtons();
  initChart();
  renderCurrentChart();
  refreshData();

  refreshTimer = setInterval(refreshData, CONFIG.refreshMs);
  weatherTimer = setInterval(periodicRefreshExternalWeather, CONFIG.weatherRefreshMs);
})();
```